

# Group 70 Progress Report: Detection of SMS Spam Phishing

Gayan Athukorala, Raymond Ma, Alexander Yoon

{athukorg,mar47,yoona1}@mcmaster.ca

## 1 Introduction

SMS phishing texts are an issue that millions of people face everyday, texts ranging from unpaid tolls, a package you forgot to pick up, or a major bank telling you they froze your cards. Unfortunately, many Canadians say that these phishing texts are only becoming more common. In 2023, Canadians lost nearly 24 million dollars to these spam phishing texts, which according to the Canadian Anti-Fraud Centre is a drop in the bucket, as less than 10% of victims report their fraud. As scammers develop more and more robust techniques to deceive unsuspecting individuals, these numbers will only increase and the most vulnerable members of our society such as young teens and the elderly will be at risk of data breaches, fraud, and theft (Hudes, 2024). That is why spam and phishing detection is ever more important, being able to automatically determine if a text that you've received is legitimate or phishing could protect millions of people.

Mobile operating systems do some screening, but adversaries move quickly. The content of a spam phishing attack can evolve hourly, borrow legitimate language from real businesses, and send messages from thousands of programmatically created accounts (Taylor et al., 2007). Community groups and small organizations rarely have the budget for commercial filters, yet they are often the first to feel the impact when staff or volunteers are targeted (Chernicky, 2025). This is why our team is investing in tools that are lightweight, open, and easy to explain. We want a detector that pairs strong accuracy with plain-language reasoning so that other developers, instructors, or security leads can audit and adapt it without guesswork.

For our 4AL3 project, we created a small end-to-end pipeline that lives inside the `model_code` folder. This report walks through the dataset, feature design, implementation, experiment, and anal-

ysis we ran, and behaviour we observed.

## 2 Related Work

Early spam detection was typically based on manual hardcoded rules, such as keyword lists, blacklists, and header analysis to determine spam scores. While temporarily effective, these systems were brittle and needed to be manually updated to keep up with evolving spam tactics.

With machine learning, models could automatically learn patterns and features from text to determine spam likelihood. Naive Bayes and SVM classifiers became standard approaches for both email and SMS spam detection because of their simplicity and effectiveness on bag-of-words representations. Other studies found that approaches such as Random Forests and AdaBoost could achieve near-perfect accuracy on benchmark datasets. Unfortunately, the models relied on surface level features and were not able to detect spam messages that appeared normal on the surface but carried a malicious intent.

Recent methods of spam detection involve using deep learning to understand the semantic meaning of messages. Recurrent Neural Networks (RNNs) and Transformer-based architectures have been applied to capture contextual information and subtle linguistic cues that distinguish spam from legitimate messages. These models can learn complex patterns and adapt to new spam tactics, but they require large amounts of labelled data and computational resources to train effectively (Wang, 2025).

One of the biggest issues we found in our first iteration of this project was the class imbalance in the dataset. Since spam messages are much rarer than legitimate messages and as phishing techniques are constantly evolving, our team was not confident that our model would be able to generalize well to unseen and real-world data that goes beyond our dataset. We took a look at how industry profession-

als have approached this imbalance to train robust spam phishing detectors and found that using data augmentation techniques such as synonym replacement, word embeddings, and back-translation have been shown to improve model generalization on imbalanced datasets (Regina et al., 2020).

For our final project, our team uses a linear SVM with a hybrid feature set combining TF-IDF n-grams and handcrafted behavioural cues. This approach balances interpretability, efficiency, and performance, allowing us to establish a strong baseline while retaining the ability to reason about feature importance and model decisions.

### 3 Dataset

We use the public SMS Spam Collection dataset from de Almeida and Hidalgo (2011), which contains 5,574 messages labeled as either spam or ham. In this context, spam refers to unwanted or malicious SMS messages designed to deceive recipients—such as prize alerts, missed delivery scams, or fake bank notifications—while ham refers to legitimate, non-spam messages that represent normal conversation.

Out of the full corpus, 747 messages (13.4%) are labeled as spam and 4,827 messages (86.6%) as ham. To preserve this ratio across all splits, we apply stratified sampling to divide the data into 70% training (3,902 messages), 10% validation (558 messages), and 20% test (1,114 messages).

Before feature extraction, we perform the following preprocessing steps:

- **Label encoding:** converting ‘spam’ and ‘ham’ labels into binary format (1 for spam, 0 for ham).
- **Lowercasing:** converting all text to lowercase to ensure uniformity.
- **URL masking:** replacing web links with a <URL> placeholder to focus on message content.
- **Stratified splitting:** dividing the dataset into 70%/10%/20% splits while maintaining the overall spam/ham ratio.

Figure 1 shows that while most messages are short, spam texts tend to be longer due to promotional or deceptive phrasing. This check confirms that the training and evaluation splits each contain a representative range of message lengths and content styles.

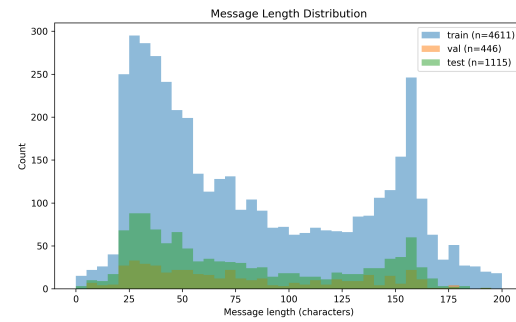


Figure 1: Message length distribution across train, validation, and test splits. The mix of short conversational texts (ham) and longer scam pitches (spam) remains consistent in each split.

Since the progress report, our team implemented data augmentation to address the class imbalance mentioned above, and improve the model’s ability to generalize. The following augmentation techniques were applied to spam messages in the training split to synthetically increase their number:

- **Synonym Replacement:** Words in the spam messages were replaced with their synonyms using WordNet to create varied versions of the same message.
- **Contextual Word Embeddings:** Using a transformer-based model (DistilBERT), we replaced certain words in the spam messages with contextually similar words predicted by the model. Unlike simple synonym replacement, this method considers the surrounding context to select intelligent substitutes.
- **Back Translation:** This technique involves translating the spam messages to another language (German in our case), and then translating them back to English using Facebook’s WMT19 translation models. This process is more resource intensive, but it generates paraphrased versions of the original messages that retain their meaning while altering the message structure more than the other two techniques.

Each augmentation technique was applied to a batch of 200 spam messages from the training set, resulting in an additional 600 synthetic spam messages. These augmented messages were then combined with the original training data to create a more balanced dataset for model training.

## 4 Features

After preprocessing, we represent each message with a hybrid mix of lexical and structural cues. The augmented training split now contains 4,611 messages (1,131 spam / 3,480 ham) after adding 600 synthetic spam texts, but we keep the same feature template so results stay comparable. The lexical block uses a TF-IDF vectorizer over unigrams and bigrams with a 3,000-term vocabulary, capturing words and short phrases that separate spam from ham. The structural block is a manually crafted set of seven features that capture formatting habits often seen in phishing messages. These include:

1. Character length
2. Word count
3. Digit count
4. Uppercase letter ratio
5. URL count (frequency of <URL> placeholder)
6. Currency flag (detects currency codes or symbols)
7. Exclamation mark count

We concatenate the 3,000-dimensional TF-IDF vector with the seven dense structural features (standardized in the pipeline), producing a 3,007-dimensional representation for each message. The code now trains three variants—TF-IDF only, handcrafted only, and the combined hybrid—so the ablations below can show how much each component contributes. Digit-heavy cues (e.g., number counts and currency hints) and emphasis markers (uppercase, exclamation points) target short codes and payment requests, while length and URL counts flag templated promotions without relying on exact wording.

## 5 Implementation

We use a Linear Support Vector Machine (SVM) with scikit-learn’s LinearSVC implementation as our spam/ham classifier. Linear SVMs are fast to train and evaluate, and their weights are easy to interpret, which aligns with our goal of building an explainable model.

The loss function we used was a squared hinge loss, which is default in LinearSVC. This loss function penalizes misclassifications more heavily

as they move further from the decision boundary, which helps the model focus on correctly classifying difficult examples.

$$L(w, b) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w \cdot x_i + b))^2$$

We set the regularization parameter  $C$  to 10.0 to balance fitting the training data and maintaining generalization to unseen data. We also set `class_weight='balanced'` to account for the class imbalance in the dataset, ensuring that spam messages receive higher weight during training. The optimization technique used is coordinate descent (also default for LinearSVC), which updates/optimizes along a single variable at a time while keeping the others fixed until it reaches convergence. This efficiently finds the optimal weights for the linear decision boundary.

Classical SMS baselines on this corpus use Multinomial Naive Bayes or linear SVMs and report mid-90s accuracy/F1 with bag-of-words alone (Hidalgo et al., 2006). Our implementation keeps the same lightweight Linear SVM backbone but extends it with structural features and augmented spam examples to lift recall without sacrificing speed or interpretability.

Comparing this against a simple majority baseline (predicting all messages as ham) yields 87.9% validation accuracy and 85.5% test accuracy, with zero precision, recall, or F1 for the spam class.

Retraining on the augmented dataset gives:

- Train: 99.91% accuracy, 99.82% precision, 99.82% recall, 99.82% F1 (tn=3,479, fp=1, fn=2, tp=1,129)
- Validation: 98.65% accuracy, 92.86% precision, 96.30% recall, 94.55% F1 (tn=388, fp=4, fn=2, tp=52)
- Test: 98.56% accuracy, 93.97% precision, 96.30% recall, 95.12% F1 (tn=942, fp=11, fn=6, tp=156)

Feature variants in the code now include TF-IDF only, handcrafted only, and the combined hybrid (all ablation scores below are from the validation split):

- TF-IDF only baseline: 98.43% validation accuracy, 93.69% F1.

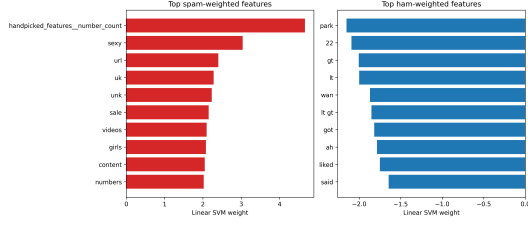


Figure 2: Top learned weights. Words about prizes, cash, and urgent action lean toward the spam side. Everyday acknowledgements lean toward the ham side.

- TF-IDF + single number\_count: 99.33% validation accuracy, 97.20% F1; full hybrid with all seven structural features: 99.10% validation accuracy, 96.30% F1. Leaving out number\_count drops F1 to 94.34%.
- Handcrafted-only: best single number\_count reaches 96.64% validation accuracy, 85.98% F1; all seven structural features together reach 97.53% validation accuracy, 90.09% F1.

The hybrid models beat both the majority baseline and the TF-IDF-only baseline while keeping the lightweight Linear SVM pipeline.

Figure 2 highlights which features pull the model toward a spam decision and which lean toward the safe side. The plot is easy to scan and reassures us that the model is picking up the same cues a human reader would notice.

## 6 Results and Evaluation

We evaluate the Linear SVM using five-fold stratified cross-validation on the augmented training split (4,611 messages, 24.5% spam). Each fold trains on four-fifths of the data and reports accuracy, precision, recall, and F1 on the held-out fifth. The mean cross-validation scores are 0.987 accuracy, 0.980 precision, 0.966 recall, and 0.973 F1 (Figure 3), with fold accuracies ranging from 0.982 to 0.992 and recall spanning 0.956 to 0.991. After cross-validation, we refit a final model on the full training split and score it once on the validation and test sets, keeping the test labels untouched until the end.

The five stratified folds preserve the updated spam/ham balance in every slice, keeping each validation subset representative. As shown in Figure 4, false positives per fold range from 3 to 7 and false negatives from 2 to 10, and the averaged fold confusion matrix is  $tn \approx 691.6$ ,  $fp \approx 4.4$ ,  $fn \approx 7.6$ ,  $tp \approx 218.6$ . Averaging these scores provides a sta-

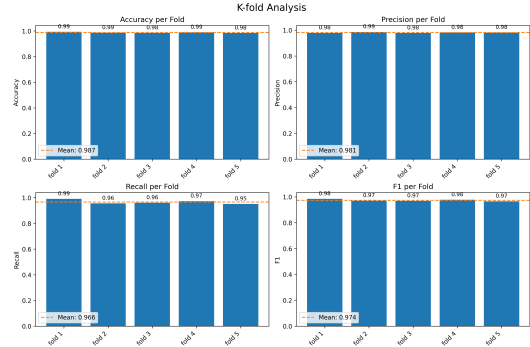


Figure 3: Cross-validation results. Bars show per-fold scores and dotted lines mark the overall average. All folds track closely, indicating stable behaviour.

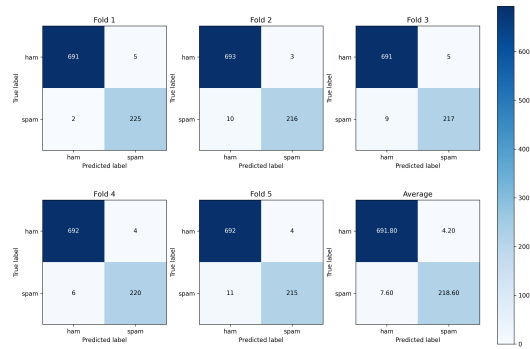


Figure 4: Confusion matrices for each validation fold and the averaged result. True negatives dominate and false positives remain low across all folds.

ble view of variance without starving any fold of spam examples.

Confusion matrices for the final model (Figures 5–7) mirror the cross-validation averages: train  $tn = 3,479$ ,  $fp = 1$ ,  $fn = 2$ ,  $tp = 1,129$ ; validation  $tn = 388$ ,  $fp = 4$ ,  $fn = 2$ ,  $tp = 52$ ; test  $tn = 942$ ,  $fp = 11$ ,  $fn = 6$ ,  $tp = 156$ . True negatives dominate while spam recall stays above 0.96 on both validation and test splits.

Across folds and held-out splits, true positives and true negatives dominate and false predictions stay in the single digits. The close match between the averaged fold matrix and the validation/test confusion matrices shows that the cross-validation strategy remains reliable after augmentation, and reporting accuracy, precision, recall, and F1 continues to capture both overall correctness and spam sensitivity.

### 6.1 Feature Ablation Study

To understand which handcrafted features actually help the model, we ran ablation experiments. We tested each feature individually and also removed

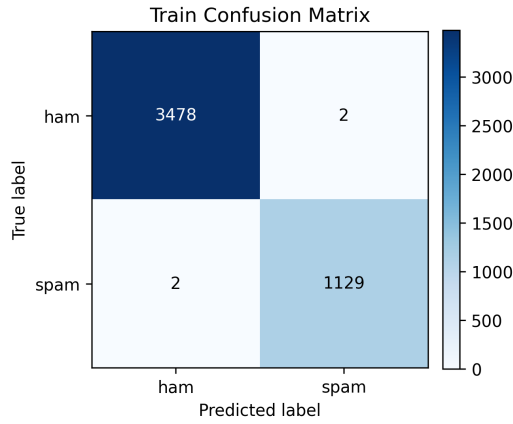


Figure 5: Confusion matrix on the full training split. The model captures nearly all spam while preserving legitimate messages.

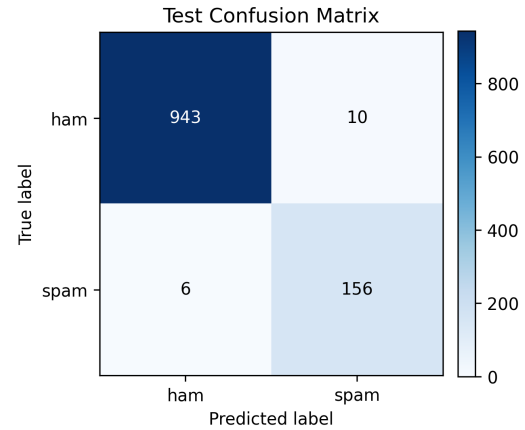


Figure 7: Confusion matrix on the unseen test split. Most mistakes are subtle spam messages resembling ordinary conversation.

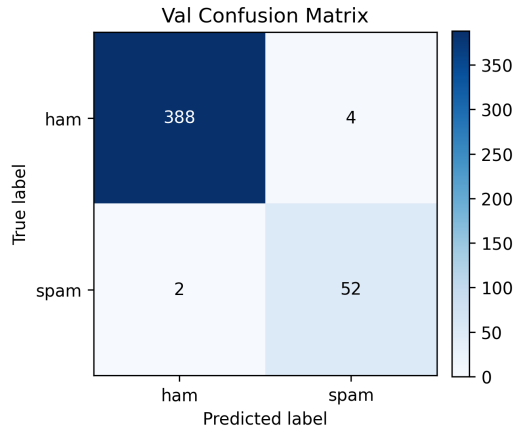


Figure 6: Confusion matrix on the validation split. The model maintains a strong balance between detecting spam and avoiding false alarms.

each feature one at a time from the full set. This reveals two things: how much a feature contributes when added, and how much performance drops when that feature is removed.

Figure 8 shows results when TF-IDF is included. The TF-IDF-only baseline scores 0.937 F1 (98.43% validation accuracy). Adding `number_count` provides the largest boost, reaching 0.972 F1 and 99.33% accuracy, while single additions like `exclamation_count` fall below the baseline (0.919 F1). Leave-one-out runs follow the same pattern: removing `number_count` drops F1 to 0.943, while the full hybrid with all seven features reaches 0.963 F1.

Figure 9 removes TF-IDF and uses only the handcrafted features. The `number_count` feature alone scores 0.860 F1 with 96.64% validation accuracy, nearly matching the full handcrafted set

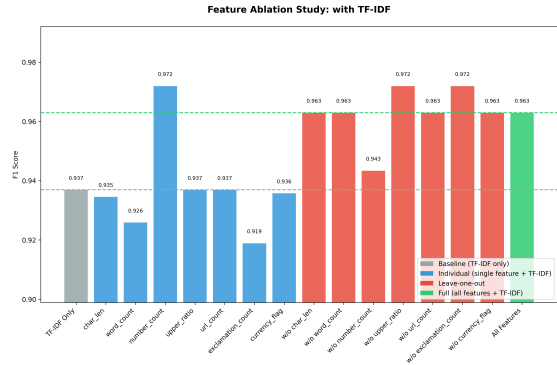


Figure 8: Feature ablation with TF-IDF. Blue bars show each handcrafted feature added to TF-IDF. Red bars show performance when one feature is removed from the full set. Grey bar is TF-IDF alone.

at 0.901 F1 and 97.53% accuracy. Meanwhile, `currency_flag` on its own scores just 0.036 F1 and `url_count` reaches only 0.138 F1, showing these cues need lexical context to be useful.

## 7 Progress

Our progress report plan was to clean the SMS Spam Collection data, build a TF-IDF + Linear SVM baseline with a handful of structural cues, and report stratified validation metrics. We completed that pipeline, produced the hybrid feature set, and generated fold-level metrics and plots for the report. Since then we added 600 augmented spam messages to rebalance the training split (raising it to 4,611 examples), ran full feature ablations (TF-IDF only, handcrafted only, and the combined hybrid), and expanded evaluation with five-fold cross-validation, confusion matrices, and updated baseline comparisons. Overall we followed the



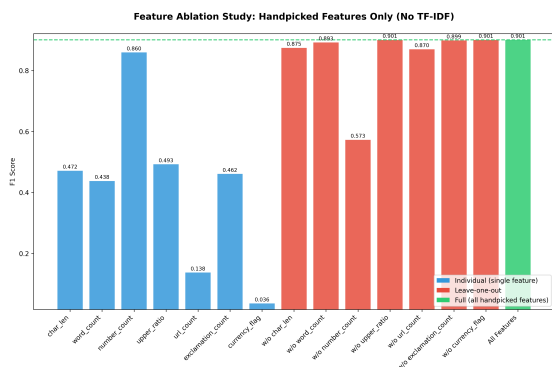


Figure 9: Feature ablation without TF-IDF. Using only handcrafted features, number\_count alone nearly matches the full set. Other features struggle on their own.

original roadmap while layering on augmentation and ablation experiments to make the final results more complete.

Compared to the initial plan, the biggest changes came from class imbalance and stakeholder feedback. The augmented spam examples were added to improve coverage of phishing styles we saw in validation, and the ablation sweeps were included to demonstrate which structural cues meaningfully help beyond TF-IDF. We also broadened evaluation from a single validation split to five stratified folds plus refreshed confusion matrices, so we could show stability rather than a single-point snapshot. These additions lengthened the experiment schedule but gave us clearer evidence that the hybrid design holds up after the dataset shift.

In practice, the team stuck closely to the division of work from the progress report: preprocessing and pipeline code landed early, while later cycles focused on interpretability plots and systematic ablations. The only scope additions were the augmentation pass and extra evaluation figures, both motivated by keeping the final report complete and consistent with the updated dataset. If time allowed, the next logical step would be stress-testing the same pipeline on an external corpus to confirm the trends we now observe internally.

## Error Analysis

Overall, our current model performs very well, achieving over 98% accuracy and an F1 score above 95% on the test set. However, there are still some misclassifications that are worth analyzing to identify potential areas for improvement. The biggest tool used to determine these misclassifi-

cations were the confusion matrices shown in the results section. From these confusion matrices, we see that the model was able to classify nearly all messages correctly, with only a handful of false positives and false negatives. The model seems to flag false positives at a slightly higher rate than false negatives.

Upon further inspection on the specific messages that the model misclassified, we found that many of the false positives were messages that without any context actually read as spam or phishing messages, but are actually ham. For example the message "Sir, I need AXIS BANK account no and bank address." was incorrectly classified as spam by the model. Although this appears as a misclassification in our data, in reality if this message was sent unprompted by an unknown number, it would likely actually be a phishing attempt, so this can be interpreted as a positive signal that the model would perform well if actually deployed and used in the real world. Conversely, the false negatives, or the spam messages that the model did not detect, were often messages that appear to be harmless or part of a normal conversation. For example, the message "Do you ever notice that when you're driving, anyone going slower than you is an idiot and everyone driving faster than you is a maniac?" was incorrectly classified as ham by the model. This message sent alone does not appear to be dangerous or spam, but could still be part of a larger phishing attempt if sent in a certain context, indicating a limitation of training on a dataset of single messages. Overall, our error analysis shows that our model performs extremely well when classifying single messages as spam or ham, but there are limitations when it comes to being able to detecting ordinary looking messages that are meant to gain the user's trust and trick them.

Comparing these results to the one's from our initial progress report in Figure 10, we see that the initial model actually performed slightly better. The initial model achieved an accuracy of 98.88% on the test set (compared to the current model's 98.56%). Beyond accuracy, the initial model has better precision while the current model has higher recall. In the context of a spam detector, our current model would be more ideal, we would rather have more false positives (harmless messages incorrectly being marked as spam) than false negatives (dangerous messages going undetected). The main difference between the two models is that the current model implemented data augmentation

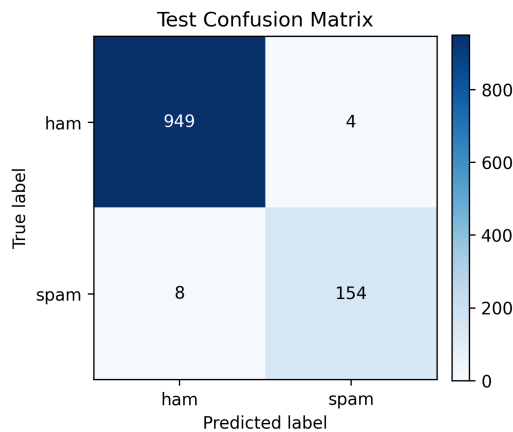


Figure 10: Confusion matrix on the Test split from the model used in the progress report.

techniques to try to improve generalization, so the discrepancy in performance could be interpreted as a trade-off between generalization and specific performance on this dataset. However, both models perform extremely well and the differences in performance are very minor.

If we were to continue working on this model, the steps that we would take to address these errors would be:

- **Collect more data:** One of the biggest limitations of our current model is that it is trained on a relatively small dataset of 5,574 messages (6174 with data augmentation). To improve the model's ability to generalize and detect more subtle phishing attempts, we would look for additional datasets to train and validate the model on.
- **Explore contextual features:** Many of the misclassifications in our error analysis were due to messages that appeared harmless on their own but could be part of a larger phishing attempt. To address this, we would explore adding contextual features such as sender information, message history, or conversation context to help the model better understand the intent behind messages.
- **Experiment with more complex models:** While our current linear SVM model performs well, we would like to experiment with more complex models such as ensemble methods or fine-tuned transformer architectures to see if we can further boost performance while still maintaining interpretability.

## Team Contributions

- **Gayan Athukorala** led dataset preprocessing and spam augmentation (synonym replacement, contextual embedding substitutions, back-translation), expanded the introduction and related work, updated the introduction, dataset section, and error analysis section.
- **Raymond Ma** implemented the feature engineering code (TF-IDF and handcrafted cues), designed and ran the feature ablation experiments, and produced the supporting experiment code, plots, and evaluation figures while adding the feature ablation section to the report.
- **Alexander Yoon** updated the features, implementation, and evaluation sections after augmentation, integrated ablation findings into those sections, wrote and revised the Progress section, and ensured consistency across the updated metrics, tables, and descriptions in the final report.

## References

- Brian Chernicky. 2025. We really need to talk about the scams targeting nonprofits. NonProfits Insurance Alliance.
- Tiago Agostinho de Almeida and Jose Maria Gomez Hidalgo. 2011. Sms spam collection. Kaggle.
- Jose M. Gomez Hidalgo, Pablo Garcia Bringas, Eva Puertas Sanz, and Eva Cernadas. 2006. [Content-based SMS spam filtering](#).
- Sammy Hudes. 2024. Overwhelmed with spam texts? stats show the problem is getting worse. Global News.
- Mehdi Regina, Maxime Meyer, and Sebastien Goutal. 2020. Text data augmentation: Towards better detection of spear-phishing emails. arXiv preprint arXiv:2007.02033.
- Bradley Taylor, Dan Fingal, and Douglas Aberdeen. 2007. The war against spam: A report from the front line. Workshop on Machine Learning in Adversarial Environments for Computer Security.
- Xinyi Wang. 2025. Spam filtering in the modern era: A review of machine learning, deep learning, and system comparisons. In *Proceedings of the 2nd International Conference on Data Analysis and Machine Learning (DAML 2024)*, pages 451–458.